
Lab Exercise 1: Implementing Abstract Data Types (ADTs) in C++

Objective:

- To understand and implement Abstract Data Types (ADTs) using C++.
 - To learn how C++ provides mechanisms to define ADTs with encapsulated data and methods.
 - To explore how constructors guarantee proper object initialization.
 - To practice separating code into header files (.h), implementation files (.cpp), and client files (main.cpp).
-

Problem:

Implement a **SimpleCalculator** class that performs basic arithmetic operations on two numbers, as described below.

The students need to implement a **SimpleCalculator** ADT in C++, which can perform basic arithmetic operations (addition, subtraction, multiplication, and division) on two numbers. The method/way of reading/having the input numbers will depend on the students' understanding.

Divide your code into separate files for better organization: a header file for the class declaration, a source file for method implementation, and a client file for the `main` function.

Instructions:

1. **Create a Header File (SimpleCalc.h)**
 - This file will contain the declaration of the `SimpleCalc` class, including private members and public method prototypes.
2. **Create a Source File (SimpleCalc.cpp)**
 - This file will contain the implementation of all the methods declared in the `SimpleCalc.h` header file.
3. **Create a Client File (main.cpp)**
 - This file will contain the `main` function where the **SimpleCalc** is tested by performing different operations.

Sample output for input 10 2 :

```
The result of addition is: 12
The result of subtraction is: 8
The result of the multiplication is: 20
The result of division is: 5
```

Error: Division by zero!

This output will appear based on the input given.

Students are allowed to use the **lecture slides only** and are **not permitted to use generative AI tools**.